



# Robotics & Computer Vision

Week 2 — Camera Model & Mapping

# Week 2 Learning Objectives

By the end of this week, you will have a solid foundation in the mathematical and practical aspects of how cameras work — essential for any vision-based robotic system.

1

## Pinhole Camera Model

Understand the simplest mathematical model of a camera and how it approximates real optics.

2

## 3D → 2D Projection

Master perspective projection: how a 3D world point maps to a 2D pixel coordinate.

3

## Intrinsic Parameters

Get familiar with the camera matrix  $K$  and the role of focal length and principal point.

4

## Hands-On Practice

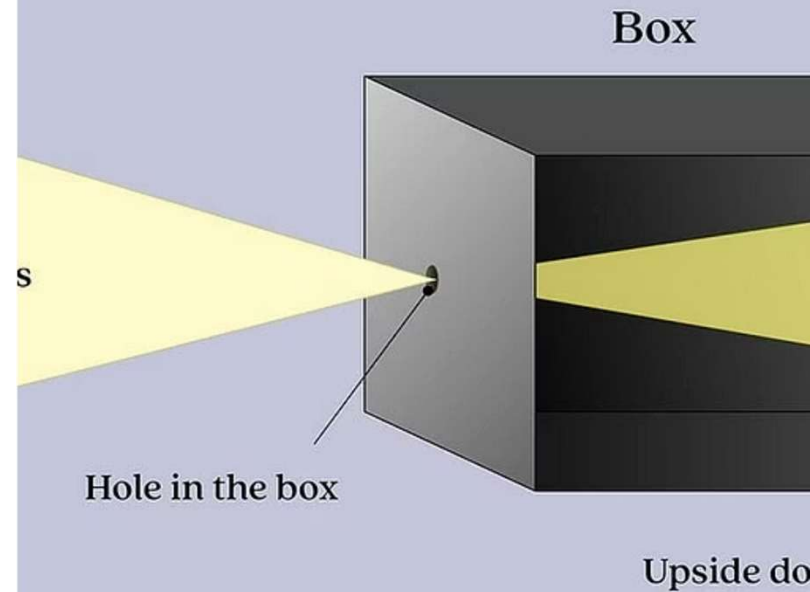
Apply calibration and projection with real cameras using OpenCV in Python.

PART 1

# The Pinhole Camera Model

Understanding how light travels through a tiny aperture to form an image — the foundational concept of all camera geometry.

## Pinhole Photography W



# What Is the Pinhole Camera Model?

The pinhole camera is the **simplest mathematical model** for describing how a camera forms an image. It abstracts a real camera down to its geometric essence, ignoring lens effects and sensor noise.

## How It Works

- A camera = a tiny **pinhole** (aperture) + an image sensor
- Light rays from the scene pass through the pinhole
- Each ray projects to exactly one point on the image plane
- The resulting image is **inverted** (upside-down and mirrored)

## Why It Matters

Despite its simplicity, the pinhole model is the basis for all camera calibration and 3D reconstruction algorithms used in robotics and computer vision. Understanding it deeply is critical before moving to more complex models.

# Key Components of a Camera

## Optical Center (C)

The point through which all light rays pass. Also called the **center of projection**. All projection lines converge here.

## Image Plane

The flat surface where the projected image is formed. In digital cameras, this corresponds to the sensor array (CCD/CMOS).

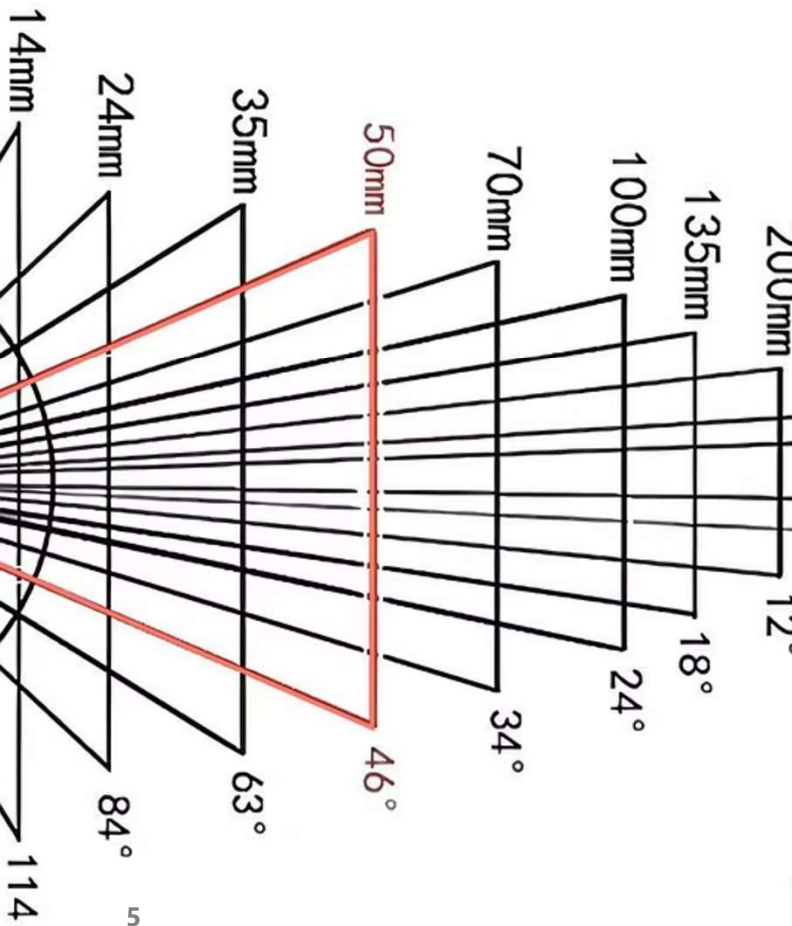
## Focal Length (f)

The perpendicular distance from the optical center C to the image plane. Controls the **zoom level** and scale of the projected image.

## Principal Point

The point where the optical axis intersects the image plane. Ideally the image center, but may be offset in real cameras.

Length unit: mm



# Coordinate Systems in Camera Geometry

Working with cameras requires understanding **three distinct coordinate frames**. Misunderstanding these leads to incorrect projections and robot errors.



## World Frame

Global 3D coordinates  $(X, Y, Z)$  — the reference frame of the physical environment where objects and the robot live.



## Camera Frame

3D coordinates  $(x_c, y_c, z_c)$  relative to the camera's optical center. Z-axis points along the optical axis.



## Image Frame

2D pixel coordinates  $(u, v)$  on the digital sensor. Origin is typically the top-left corner of the image.

# 3D to 2D Perspective Projection

The core geometric operation: given a 3D point in the camera frame, compute its 2D location on the image plane using **similar triangles**.

## Projection Equations

A 3D point  $(X, Y, Z)$  in the camera frame projects to image coordinates:

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z}$$

This is the **perspective projection equation** — the further away an object (larger  $Z$ ), the smaller it appears (smaller  $x, y$ ).

## Key Insight

The division by  $Z$  is what makes perspective projection *non-linear*. Parallel lines converge. Distant objects appear smaller. This models how the human eye and real cameras perceive depth.

Notice that if  $Z$  doubles, the projected image size **halves** — mimicking real-world perspective.

# Intrinsic Parameters

Intrinsic parameters describe properties **internal to the camera** — independent of where the camera is placed in the world. They convert metric image coordinates to pixel coordinates.

## $f_x, f_y$ — Focal Length (in pixels)

Encode the focal length scaled by the pixel size.  $f_x = F / dx$  and  $f_y = F / dy$ , where  $dx, dy$  are pixel dimensions in mm. Usually  $f_x \approx f_y$  for square pixels.

## $c_x, c_y$ — Principal Point

The pixel coordinate of the optical axis intersection. Ideally at the image center (width/2, height/2), but real cameras have small offsets due to manufacturing tolerances.

## $s$ — Skew Coefficient

Accounts for non-orthogonal pixel axes. In almost all modern cameras,  $s = 0$ . Non-zero skew appears in old cameras or when using non-standard image acquisition setups.

☐ All intrinsic parameters are stored together in the **3×3 Camera Matrix K**. Calibrating K is the first and most critical step in any vision pipeline.

# The Camera Matrix K

The intrinsic camera matrix **K** is a 3×3 upper-triangular matrix that encodes all internal optical properties of a camera. It transforms normalized image coordinates into pixel coordinates.

## Matrix Structure

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

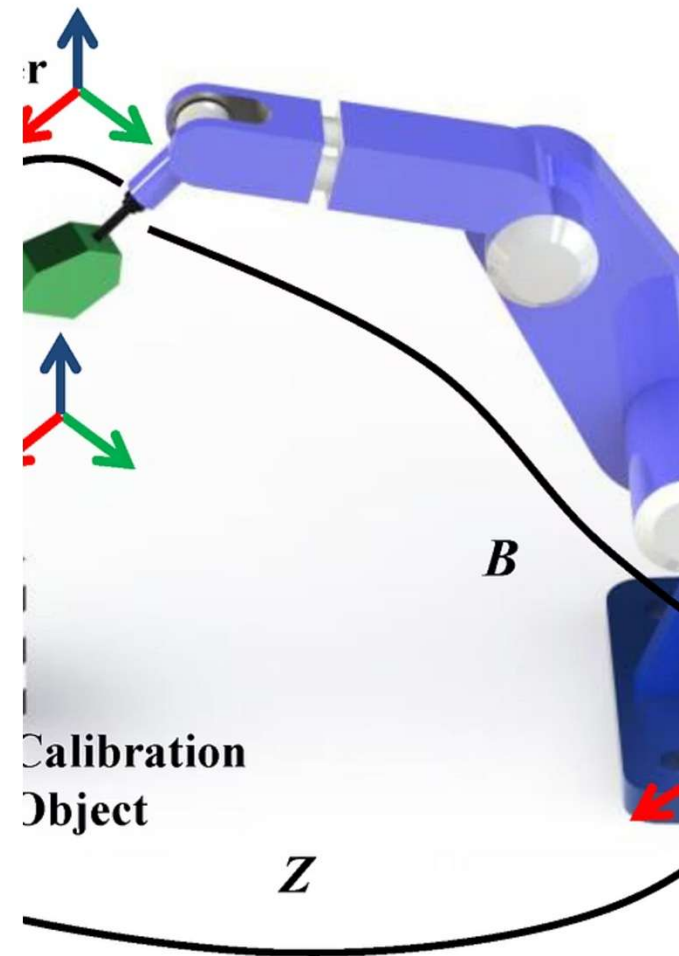
## What Each Element Does

- **$f_x, f_y$** : Scale the projection to pixel units. Controls image magnification.
- **$c_x, c_y$** : Shift the origin from optical center to top-left corner of the image.
- **$s$** : Skew — almost always 0 in practice.
- The bottom row  **$[0, 0, 1]$**  makes **K** work in homogeneous coordinates.

PART 2

# Extrinsic Parameters & Transformation

How the camera is positioned and oriented in the 3D world — bridging the gap between world coordinates and camera coordinates.



# Extrinsic Parameters

While intrinsic parameters describe the camera's internal optics, **extrinsic parameters** describe where the camera is in the world — its position and orientation relative to a fixed world frame.

## Rotation Matrix ( $R$ )

A  $3 \times 3$  orthogonal matrix encoding the **orientation** of the camera relative to the world. Rotates world coordinates into camera coordinates. Has 3 degrees of freedom.

## Translation Vector ( $t$ )

A  $3 \times 1$  vector encoding the **position** of the world origin in camera coordinates. Shifts the coordinate system from world to camera. Also 3 degrees of freedom.

## Combined Transform $[R|t]$

Together,  $R$  and  $t$  form the  **$3 \times 4$  extrinsic matrix  $[R|t]$**  that transforms any world point into the camera's local coordinate frame — essential for projection.

# World → Camera Transformation

To project a world point onto the image, we first transform it from the world frame into the camera frame using the extrinsic matrix.

## Transformation Formula

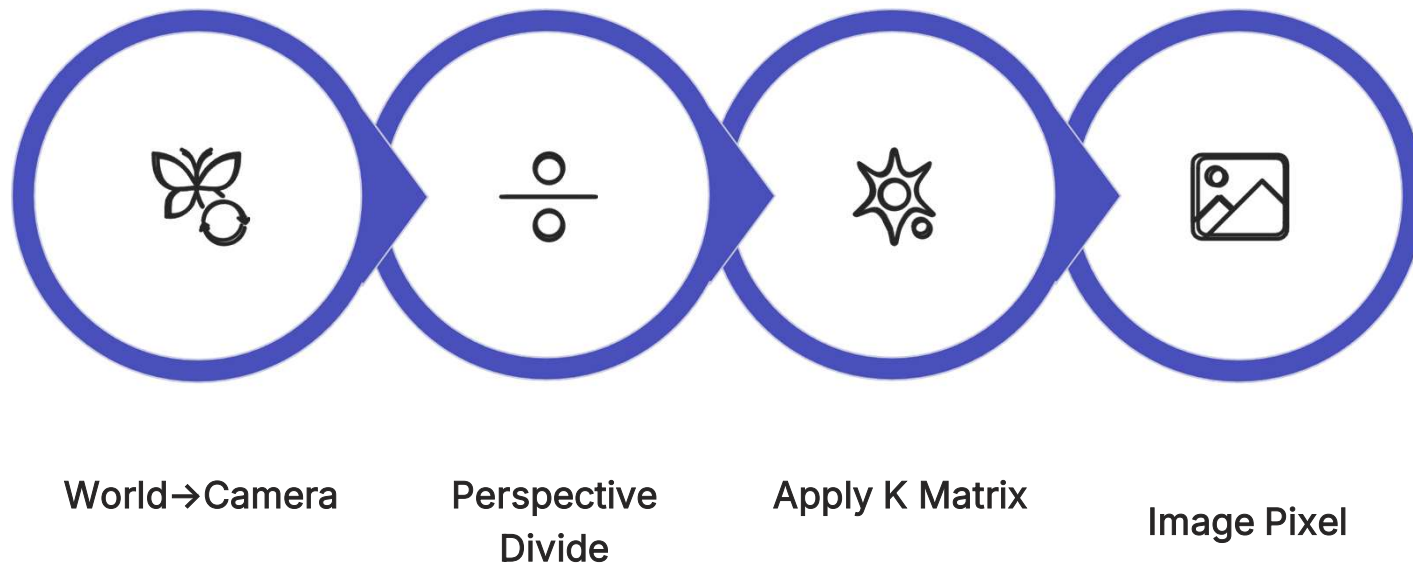
$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

## Breaking It Down

- $X_w, Y_w, Z_w$  — 3D point in world coordinates (homogeneous: append 1)
- $R$  — rotates the coordinate axes to align with the camera's orientation
- $t$  — translates the origin from world center to camera center
- $X_c, Y_c, Z_c$  — resulting point in the camera frame, ready for projection

# The Full Projection Pipeline

Putting it all together: from a raw 3D world point to a final 2D pixel coordinate, every step in the pipeline must be applied in the correct order.



Each step has a distinct role: extrinsic parameters handle **pose**, the perspective divide handles **depth**, and K handles **sensor geometry**. Errors in any step propagate to the final result.

# The Complete Projection Matrix

All transformation steps can be combined into a single compact formula using the **3×4 projection matrix P**, which directly maps a homogeneous world point to a homogeneous image point.

## Compact Formula

$$\mathbf{p} = K \begin{bmatrix} R & |t \end{bmatrix} \mathbf{x}_w$$

$$P = K \begin{bmatrix} R & |t \end{bmatrix}$$

Where **P** is the 3×4 camera projection matrix.

## Why This Matters

This single matrix multiplication replaces the multi-step pipeline. In practice:

- K encodes intrinsic (calibrated once per camera)
- $[R|t]$  changes every time the camera moves
- P is recalculated when the camera pose changes
- Used heavily in robotics for **real-time projection**

PART 3

# Lens Distortion

Real lenses are not perfect — they introduce geometric distortions that must be corrected for accurate 3D measurements and robot manipulation.

# Why Distortion Matters

The ideal pinhole model assumes perfect geometry. Real lenses — especially wide-angle and fisheye lenses — introduce **systematic distortions** that bend straight lines and misplace pixels.

## Real cameras deviate from the ideal pinhole model

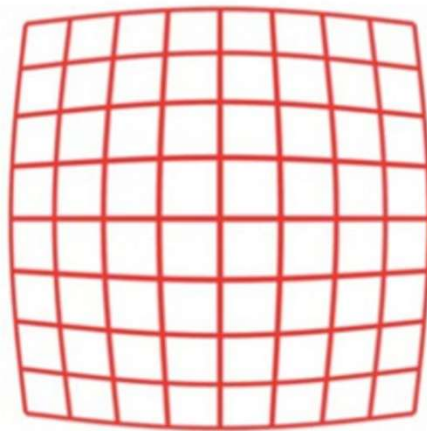
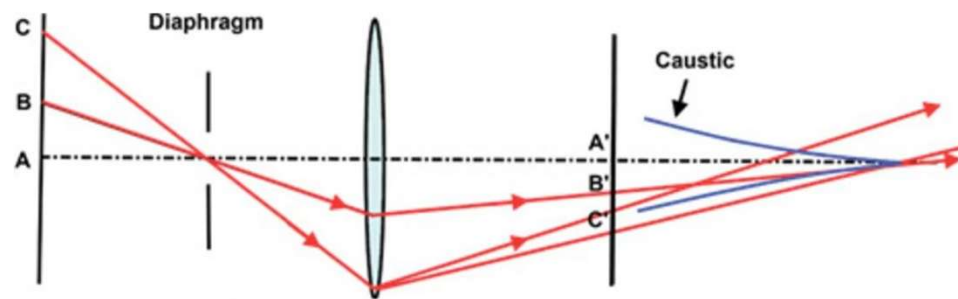
Lens curvature causes light rays to bend in non-linear ways, violating the straight-line assumption of the pinhole model.

## Distortion degrades measurement accuracy

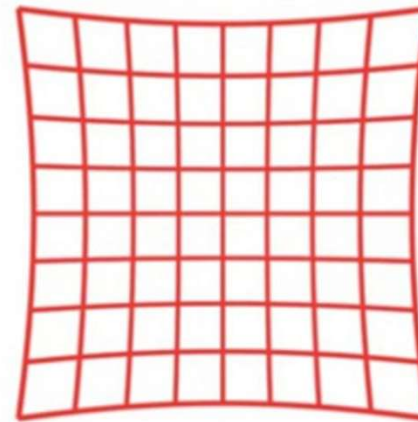
Even small amounts of distortion cause significant errors in 3D reconstruction, pose estimation, and robot picking tasks — especially near image edges.

## Calibration must correct for distortion

Before using any camera for geometric tasks, you **must** estimate and remove distortion using a calibration procedure. Skipping this step leads to unreliable results.



Barrel distortion



Pincushion distortion

# Types of Lens Distortion

## Radial Distortion ( $k_1, k_2, k_3$ )

The most common type. Caused by the spherical shape of the lens. Points are displaced **radially outward or inward** from the image center.

- **Barrel distortion** ( $k_1 < 0$ ): image bulges outward — common in wide-angle lenses
- **Pincushion distortion** ( $k_1 > 0$ ): image pinches inward — common in telephoto lenses

## Tangential Distortion ( $p_1, p_2$ )

Caused by the lens not being perfectly parallel to the image plane during assembly. Displacement is **perpendicular** to the radial direction.

- Less common than radial distortion
- Often negligible on modern cameras
- Described by coefficients  **$p_1$**  and  **$p_2$**
- Results in a slight shearing/tilting of the image

# The Distortion Model

OpenCV uses a standard parametric distortion model. These coefficients are estimated during calibration and stored alongside the K matrix.

1

## **k1, k2, k3 — Radial Coefficients**

Model the radial bending of pixels away from the center. k1 dominates at low radii, k2 at mid-range, k3 for strong wide-angle lenses. **For most cameras, k1 and k2 are sufficient.**

2

## **p1, p2 — Tangential Coefficients**

Model the misalignment between lens and sensor planes. Usually very small values. Including them improves calibration accuracy but adds complexity.

3

## **Dist Coeffs Vector**

OpenCV stores all coefficients as a single vector: **[k1, k2, p1, p2, k3]**. This is passed directly to undistortion and projection functions.

# Correcting Lens Distortion

Once the distortion coefficients are known (from calibration), we can **remove distortion** from every captured frame before further processing.

## OpenCV Undistortion

```
undist = cv2.undistort(  
    frame,    # distorted input  
    K,        # camera matrix  
    dist      # distortion coefficients  
)
```

This applies the **inverse distortion map** to remap every pixel to its correct geometric position.

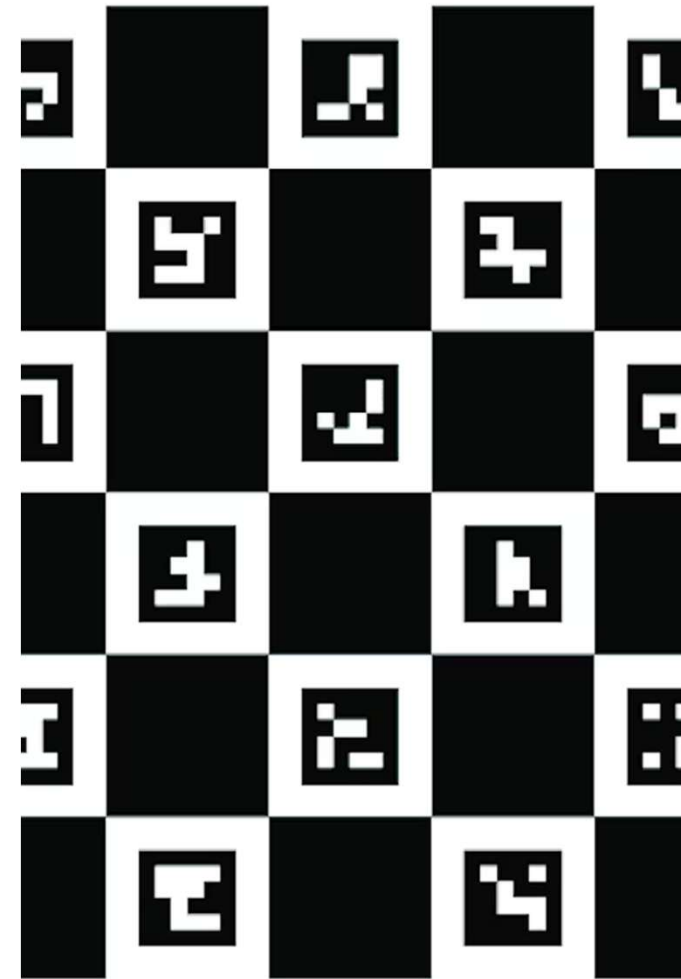
## Important Notes

- Always undistort images **before** running feature detection or projection
- For real-time systems, precompute the remap using **cv2.initUndistortRectifyMap()** for speed
- Undistortion slightly **crops the image** edges — this is normal
- Only valid if K and dist were accurately calibrated

PART 4

# Camera Calibration

The process of precisely determining a camera's intrinsic parameters and distortion coefficients — the essential foundation for accurate 3D vision in robotics.



# Purpose of Camera Calibration

Camera calibration is the process of **estimating the unknown parameters** of a camera model from a set of known 2D-3D point correspondences. Without calibration, all projection and depth calculations are unreliable.



## Determine K Matrix

Estimate  $f_x$ ,  $f_y$ ,  $c_x$ ,  $c_y$  — the four key intrinsic parameters that define how the camera maps 3D to 2D in pixel space.



## Estimate Distortion Coeffs

Find the radial ( $k_1$ ,  $k_2$ ) and tangential ( $p_1$ ,  $p_2$ ) coefficients that describe how the lens bends straight lines.



## Enable Accurate 3D→2D

With calibrated K and dist, we can accurately project 3D robot coordinates onto the image — essential for **robot picking tasks**.

# Preparing for Calibration

Good calibration requires good data. The quality of your calibration dataset directly determines the quality of your K matrix and distortion coefficients.

01

## Print a Chessboard Pattern

Use a standard chessboard (e.g., 9×6 inner corners). Print on flat, rigid cardboard — a warped board will give poor results.

03

## Ensure Good Lighting

Avoid shadows, reflections, and motion blur. The chessboard corners must be **sharply visible** in every image for reliable corner detection.

02

## Capture from Multiple Angles

Take **at least 10–20 images** covering different positions, rotations, and distances. Include corners, tilts, and different depths.

04

## Verify Corner Detection

Use `cv2.findChessboardCorners()` to confirm all corners are detected in each image before running calibration.

# Camera Calibration with OpenCV

OpenCV's **calibrateCamera()** function implements Zhang's calibration method — the industry standard for estimating camera intrinsics and distortion from a planar pattern.

## Calibration Code

```
ret, K, dist, rvecs, tvecs = \
    cv2.calibrateCamera(
        objpoints,  # 3D world points
        imgpoints,  # 2D image points
        img_size,   # (width, height)
        None, None  # initial K, dist
    )
```

## Output Parameters

- **ret**: RMS reprojection error (should be < 1.0 px)
- **K**: 3×3 camera matrix with  $f_x$ ,  $f_y$ ,  $c_x$ ,  $c_y$
- **dist**: distortion coefficients [ $k_1$ ,  $k_2$ ,  $p_1$ ,  $p_2$ ,  $k_3$ ]
- **rvecs**, **tvecs**: rotation and translation for each calibration image (extrinsics)

PART 5

# Hands-On Practice with OpenCV

Nine progressive lab exercises building from basic camera capture to full calibration, projection, and robot hand-eye applications.

Joseph Howse, Prateek Joshi,  
Michael Beyeler

## OpenCV: Computer Vision Projects with Python Learning Path

Get savvy with OpenCV and actualize cool computer  
vision applications



Packt

# Lab 1: Capture Frames from Camera

The starting point for all vision work — opening a camera device, reading frames, and displaying them. This forms the **basic I/O loop** used in every real-time vision application.

## Code

```
import cv2

cap = cv2.VideoCapture(0)
ret, frame = cap.read()

if ret:
    cv2.imshow('Frame', frame)
    cv2.waitKey(0)

cap.release()
cv2.destroyAllWindows()
```

## Key Concepts

- **VideoCapture(0)**: opens the first available camera (use 1, 2 for additional cameras)
- **cap.read()**: returns (success\_flag, frame) — always check ret before using frame
- **frame**: a NumPy array of shape (H, W, 3) in BGR color format
- For continuous video, wrap in a **while True** loop and check for key press

## Lab 2: Construct the K Matrix

Before calibrating, understand how to manually construct K from known or estimated parameters. This is also useful when you have a camera spec sheet with focal length in mm.

### Code

```
import numpy as np

fx, fy = 800, 800 # pixels
cx, cy = 320, 240 # image center

K = np.array([
    [fx, 0, cx],
    [0, fy, cy],
    [0, 0, 1]
], dtype=float)
```

### Practical Notes

- For a 640×480 image, a reasonable starting estimate is  $cx=320, cy=240$
- $fx \approx fy$  for most cameras (square pixels)
- A typical webcam has  $fx \approx 600\text{--}800$  px; a phone camera may have  $fx > 1000$  px
- Always validate K against actual calibration — manual estimates are rough

# Lab 3: Project a 3D Point to 2D

The core operation: given a 3D point in camera coordinates, compute its pixel location using the camera matrix  $K$ .

## Code

```
import numpy as np

X_3d = np.array([1.0, 2.0, 5.0])
# Perspective divide first
x_norm = X_3d[0] / X_3d[2]
y_norm = X_3d[1] / X_3d[2]

p_h = np.array([x_norm, y_norm, 1])
p_2d = K @ p_h

u = int(p_2d[0])
v = int(p_2d[1])
```

## Understanding the Steps

- Divide X, Y by Z: this is the **perspective divide** giving normalized coordinates
- Multiply by  $K$ : converts to pixel coordinates ( $u$ ,  $v$ )
- The point (1, 2, 5) is 1m right, 2m down, and **5m away** from the camera
- Try changing Z — notice how ( $u$ ,  $v$ ) changes as the object moves closer/farther

# Lab 4: Load Calibration Data

In production robotics pipelines, calibration data is calibrated once and **saved to disk**, then loaded at runtime. This avoids repeating calibration every session.

## Code

```
import pickle

# Load saved calibration
with open('calib.pkl', 'rb') as f:
    calib_data = pickle.load(f)

K = calib_data['K']
dist = calib_data['dist']

print("K matrix loaded:")
print(K)
```

## Best Practices

- Save K, dist, image\_size, and reprojection error together in one file
- Use **pickle** for Python objects or **np.savez()** for NumPy arrays
- Also consider saving rvecs/tvecs if you need pose data later
- Recalibrate when the camera lens is changed, refocused, or physically adjusted

# Lab 5: Undistort an Image

Apply the estimated distortion model to produce a geometrically corrected image — a prerequisite for any accurate measurement or 3D reconstruction.

## Code

```
import cv2

# Simple undistortion
undist = cv2.undistort(frame, K, dist)
cv2.imshow('Original', frame)
cv2.imshow('Undistorted', undist)

# Faster for real-time use:
mapx, mapy = cv2.initUndistortRectifyMap(
    K, dist, None, K,
    (w, h), cv2.CV_32FC1)
undist = cv2.remap(frame, mapx, mapy,
    cv2.INTER_LINEAR)
```

## What to Observe

- With strong barrel distortion (wide-angle lens), straight lines near the edges become visibly straighter after undistortion
- The image may show **black borders** at the edges — this is expected and can be cropped
- For video, precompute **mapx, mapy** once and reuse `cv2.remap()` for each frame

# Lab 6: Draw Projected Points on Image

Visualize the result of 3D projection by drawing projected points directly onto camera frames. This is the standard technique for **verifying calibration accuracy**.

## Code

```
img = frame.copy()

projected_points = [(u1,v1),(u2,v2)]

for (u, v) in projected_points:
    cv2.circle(
        img,
        (u, v),    # center pixel
        5,         # radius
        (0,255,0), # green color (BGR)
        -1         # filled circle
    )

cv2.imshow('Projected', img)
```

## Visualization Tips

- Use **green circles** for projected points and **red** for detected points — compare them to assess calibration quality
- Use `cv2.putText()` to label each point with its 3D coordinates for debugging
- The distance between projected and detected points is the **reprojection error**
- Good calibration: reprojection error < 1 pixel

# Lab 7: Determine Object Grasp Position

The ultimate goal in robot vision: detect an object in the image and determine its 3D position for the robot arm to grasp. This combines all previous concepts.

01

## Detect Object in Image

Use color segmentation, thresholding, or a detector to find the object's pixel region in the 2D image frame.

03

## Convert to 3D (requires depth)

Using a depth camera (RGBD) or stereo vision, retrieve the depth  $Z$  at  $(u, v)$ . Then:  $X = (u - c_x) * Z / f_x$ ,  $Y = (v - c_y) * Z / f_y$ .

02

## Compute 2D Centroid ( $u_{obj}$ , $v_{obj}$ )

Calculate the center of the detected object mask using moments:  $c_x = M10/M00$ ,  $c_y = M01/M00$ .

04

## Output Robot Target ( $X$ , $Y$ , $Z$ )

The resulting 3D point in camera frame is transformed to the robot base frame via the hand-eye transform, giving the **grasp target position**.

# Lab 8: Estimate Extrinsic $[R|t]$ with solvePnP

`cv2.solvePnP()` solves the Perspective-n-Point problem: given N known 3D points and their 2D projections, estimate the camera pose (R and t).

## Code

```
import cv2

ret, rvec, tvec = cv2.solvePnP(
    objpoints,    # Nx3 world points
    imgpoints,    # Nx2 image points
    K,            # camera matrix
    dist,         # distortion coeffs
)

# Convert rotation vector to matrix
R, _ = cv2.Rodrigues(rvec)
```

## Key Notes

- **rvec**: rotation as a compact 3D vector (Rodrigues form) — not a matrix!
- **cv2.Rodrigues()**: converts  $rvec \leftrightarrow 3 \times 3$  rotation matrix R
- Requires at least **4 non-coplanar point correspondences**
- Used heavily in AR marker tracking and robot pose estimation
- Result: camera position and orientation relative to the calibration object

# Lab 9: Build a Calibration Dataset

Create a complete, high-quality calibration dataset from scratch — the professional workflow used in real robotics deployments.

1

## Capture 20 Chessboard Images

Move the board to many angles, distances, and positions.  
Cover the full image field including corners and edges.

2

## Detect Corners

Use `cv2.findChessboardCorners()` followed by `cv2.cornerSubPix()` for sub-pixel accuracy. Only keep frames where all corners are found.

3

## Run `calibrateCamera()`

Pass all valid (objpoints, imgpoints) pairs. Check that the RMS reprojection error is below **1.0 pixel**.

4

## Save K and dist Coefficients

Persist results to **calib.pkl** or **.npz** file for use in all subsequent vision pipelines and robot tasks.

# Computing the Viewing Ray

The **inverse operation** of projection: given a pixel (u, v) on the image, compute the 3D ray in camera space that passes through that pixel. Essential for depth estimation and ray-casting.

## Ray Computation

$$x_{norm} = \frac{u - c_x}{f_x}$$

$$y_{norm} = \frac{v - c_y}{f_y}$$

Ray direction vector:  $[x_{norm}, y_{norm}, 1]$

Normalize:  $d = ray / ||ray||$

## Applications

- **Depth from stereo:** rays from left and right cameras triangulate 3D position
- **Ray casting in 3D:** find which 3D object a mouse click selects in a scene
- **Back-projection:** lift 2D detections to 3D rays for multi-view fusion
- **Undistortion:** compute undistorted pixel positions from distorted ones

# Depth Estimation via Stereo Vision

A single camera cannot measure depth directly — you need either a depth sensor or two cameras. **Stereo vision** mimics human binocular vision to recover depth from two synchronized cameras.

## Stereo Depth Formula

$$Z = \frac{f}{d} B$$

Where:

**f** = focal length (pixels)

**B** = baseline distance between cameras

**d** = disparity (pixel offset between left and right images)

## Stereo Pipeline

- Mount two cameras with a known **baseline B** (horizontal separation)
- Rectify both images so corresponding points lie on the **same horizontal scan line**
- Compute **disparity map** using block matching (cv2.StereoBM) or semi-global matching
- Apply depth formula pixel-by-pixel to get a full **depth image**
- Accuracy improves with larger baseline and better texture in the scene

# Hand-Eye Calibration

Hand-eye calibration determines the **rigid transformation  $T_{\text{hand\_eye}}$**  between the camera frame and the robot's end-effector frame — enabling the robot to act on what the camera sees.

## Eye-in-Hand Configuration

Camera is mounted **on the robot arm**. As the arm moves, the camera moves with it.  $T_{\text{hand\_eye}}$  is fixed; the world-to-camera transform changes with robot pose.

## Eye-to-Hand Configuration

Camera is **fixed externally**, observing both the robot and the workspace. The camera-to-robot transform  $T_{\text{hand\_eye}}$  is constant; no need to recompute as the arm moves.

## Calibration Procedure

Move the robot to multiple known poses while recording both the robot kinematics and the camera's observation of a fixed calibration target. Solve for  $T_{\text{hand\_eye}}$  using `cv2.calibrateHandEye()`.

# Mini Project: Full Camera Calibration

Bring together everything from Week 2 in a complete end-to-end calibration and projection pipeline. This mini-project is your chance to validate your understanding on real hardware.



## Calibrate Your Camera

Capture 20+ chessboard images from diverse angles, run `calibrateCamera()`, and verify RMS error  $< 1.0$  px.



## Test 3D Point Projection

Define 5–10 known 3D points, project them using  $K$ , draw on the captured frame, and visually verify they land in the correct locations.



## Save K and Dist Coefficients

Persist calibration results to a `.pkl` file. Include  $K$  matrix, dist vector, image size, and RMS reprojection error.



## Compare with Real Image

Compare projected vs. detected positions. Compute reprojection error per point and discuss where errors are largest and why.

# Homework & Reading Assignments

## Lab Work (Due before Week 3)

- Complete all **9 lab exercises** from today's session
- Submit code + output images (or screenshots) for each lab
- Build your **calibration dataset** with at least 20 images
- Run `calibrateCamera()` and report your RMS reprojection error

## Reading & Preparation

- Read **Szeliski, Chapter 2: Image Formation** — covers pinhole, lenses, sensors, and color
- Optional: Hartley & Zisserman, Chapter 6 — Camera Models for deeper theory
- Explore **OpenCV camera calibration tutorial** online documentation
- Prepare for Week 3: **Image Processing & Feature Extraction** (filtering, thresholding, centroid detection)

# Common Problems & Solutions

These are the most frequently encountered issues in Week 2 labs — know them before you hit them in practice.

## Chessboard Not Detected

**Symptom:** `cv2.findChessboardCorners()` returns False.

**Fix:** Increase image contrast, improve lighting, ensure the full board is visible with a white border around it. Try converting to grayscale and applying histogram equalization.

## Poor Calibration Accuracy (High RMS)

**Symptom:** Reprojection error  $> 2-3$  pixels.

**Fix:** Add more images from diverse angles. Ensure the board covers all regions of the image including corners. Remove outlier images with high individual reprojection error.

## Excessive Distortion

**Symptom:** Undistorted image looks warped or overcorrected.

**Fix:** Verify you are using the correct camera — check VideoCapture index. Some cameras auto-apply undistortion in firmware; check camera settings.

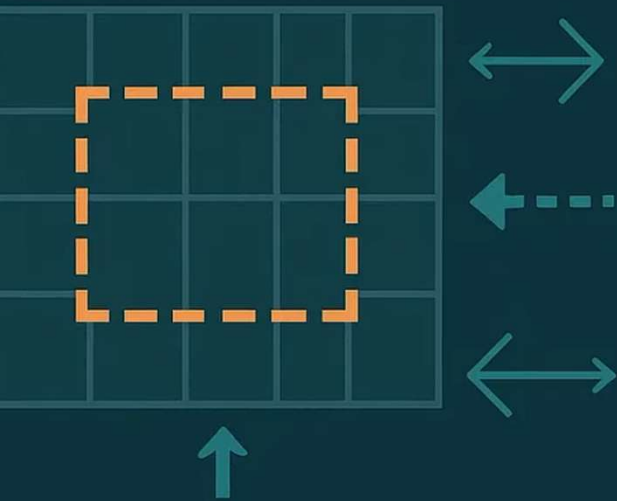
## Incorrect Projection Results

**Symptom:** Projected points land in the wrong location.

**Fix:** Double-check focal length units (pixels vs. mm). Verify the coordinate frame convention (OpenCV uses right-hand, Z forward). Check that you apply perspective divide before multiplying by K.

# Processing

Top, Filtering, Edge Detection, Vision



## Coming Up: Week 3

### Image Processing & Feature Extraction

#### Filtering

Gaussian blur, median filter, morphological operations — reduce noise and enhance structure.

#### Thresholding

Convert grayscale images to binary masks for object segmentation and region isolation.

#### Centroid Detection

Compute object centroids from binary masks using image moments — the input to Lab 7's grasp pipeline.

Contact: [phong.ledinh@hoasen.edu.vn](mailto:phong.ledinh@hoasen.edu.vn) — reach out with questions before Week 3 begins.